

CREDIT CARD APPROVAL PREDICTION

Project Description:

Banks and financial institutions receive thousands of credit card applications every day. Evaluating each application manually is a challenging and time-consuming task that often results in delays and inconsistencies. Traditional approval systems rely heavily on manual verification, making them prone to human error and reducing operational efficiency.

The Credit Card Approval Prediction project is a Machine Learning-based web application designed to automate the credit card approval process. The system analyzes an applicant's personal, financial, and employment information to determine whether the applicant is eligible for credit card approval. By learning from historical customer data, the model can accurately predict approval decisions, helping financial institutions make faster and more reliable lending decisions.

The project begins with collecting and preprocessing the dataset by handling missing values, removing duplicate records, encoding categorical variables, scaling numerical features, and performing feature engineering. Multiple Machine Learning algorithms including Logistic Regression, Decision Tree, Random Forest, and XGBoost are trained and evaluated. Performance is measured using Accuracy, Precision, Recall, F1-Score, and Confusion Matrix to identify the best-performing model.

The selected model is then integrated into a Flask web application, where users can enter applicant details such as annual income, employment status, education level, family size, housing type, work experience, and credit history. The application instantly predicts whether the applicant is likely to be approved or rejected for a credit card.

To support scalable and cloud-based prediction services, the trained model can also be deployed using IBM Watson Machine Learning, enabling real-time prediction through a secure and user-friendly web interface.

This project demonstrates how Artificial Intelligence and Machine Learning can improve banking operations by reducing manual effort, minimizing approval errors, identifying high-risk applicants, and providing faster, more consistent credit approval decisions.

Scenarios

CREDIT CARD APPROVAL PREDICTION

Scenario 1:

A bank officer receives a new credit card application from a customer. The officer enters the applicant's details, including annual income, employment type, education, housing status, family size, and credit history into the web application. Within seconds, the trained machine learning model predicts whether the applicant is eligible for credit card approval. This significantly reduces processing time and helps employees focus on applications that require manual review.

Scenario 2:

The bank's risk management team uses the application to analyze applicants with previous loan defaults or poor repayment history. The machine learning model evaluates financial and demographic features to identify applicants who are likely to default on future payments. This allows the bank to minimize financial risks while ensuring compliance with lending policies.

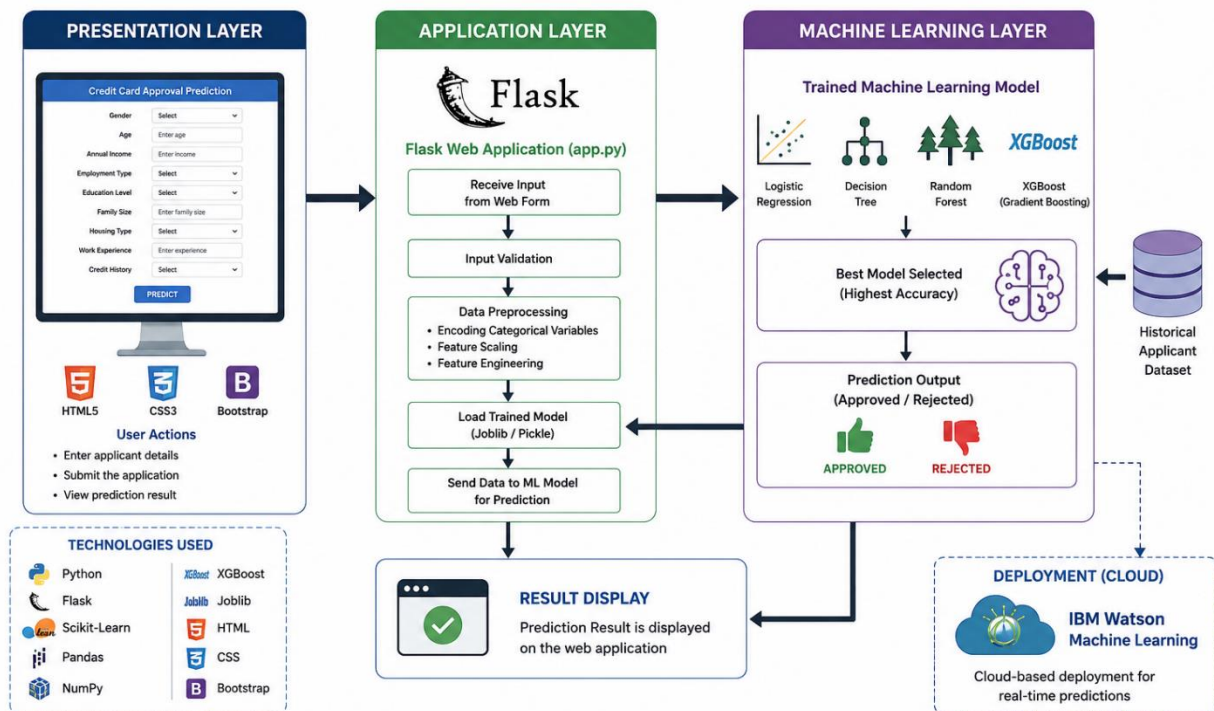
Scenario 3:

Before submitting a formal application, a customer accesses the web application to check their eligibility for a credit card. After entering employment details, annual income, credit history, and other financial information, the system instantly predicts the likelihood of approval. This helps customers understand their eligibility while reducing unnecessary application submissions.

Technical Architecture:

CREDIT CARD APPROVAL PREDICTION

TECHNICAL ARCHITECTURE CREDIT CARD APPROVAL PREDICTION



Description:

The **Credit Card Approval Prediction** system follows a modular three-tier architecture consisting of the **Presentation Layer**, **Application Layer**, and **Machine Learning Layer**. This architecture enables efficient data processing, accurate prediction, and seamless interaction between users and the machine learning model.

The **Presentation Layer** is developed using **HTML, CSS, and Bootstrap**, providing a responsive and user-friendly web interface. Users enter applicant information such as gender, age, annual income, employment type, education level, family status, housing type, work experience, family size, and credit history through the web application.

The **Application Layer** is built using the **Flask Framework**, which acts as the bridge between the user interface and the prediction model. It receives user input, validates the entered data, performs preprocessing, converts categorical values into numerical representations using the same encoding techniques applied during model training, and prepares the data for prediction.

CREDIT CARD APPROVAL PREDICTION

The **Machine Learning Layer** contains the trained classification model developed using **Scikit-Learn** and **XGBoost**. Multiple algorithms, including Logistic Regression, Decision Tree, Random Forest, and XGBoost, were trained and evaluated. The best-performing model was selected based on prediction accuracy and saved using **Joblib/Pickle**. When a user submits an application, the Flask backend loads the saved model, performs inference, and predicts whether the credit card application should be **Approved** or **Rejected**.

The prediction result is then returned to the Flask application and displayed instantly on the web interface. The application can also be deployed using **IBM Watson Machine Learning**, enabling secure cloud-based access for real-time credit card approval prediction.

(Note: If you are using database in your project definitely you need to add ER Diagram along with description)

Pre-requisites:

Before implementing this project, the following knowledge and software are required:

- Python Programming
- Machine Learning Fundamentals
- Data Preprocessing Techniques
- Flask Framework
- HTML & CSS
- Pandas
- NumPy
- Scikit-Learn
- XGBoost
- Matplotlib
- Git & GitHub
- IBM Watson Machine Learning
- Visual Studio Code

CREDIT CARD APPROVAL PREDICTION

- Jupyter Notebook

Project Workflow:

Milestone 1: Data Collection

- **Activity 1.1:** Collect the historical credit card applicant dataset from a reliable source and load it into the Python environment for analysis.
- **Activity 1.2:** Explore the dataset to understand the available features, target variable, and overall data structure.
- **Activity 1.3:** Identify and analyze missing values, duplicate records, and inconsistent data entries before preprocessing.
- **Activity 1.4:** Set up the development environment by installing Python, required libraries, and configuring the project workspace.

Milestone 2: Visualizing and Analysing the Data

- **Activity 2.1:** Perform Exploratory Data Analysis (EDA) to understand the distribution and characteristics of the dataset.
- **Activity 2.2:** Generate visualizations such as histograms, box plots, count plots, and correlation heatmaps to identify trends and relationships.
- **Activity 2.3:** Analyze important applicant features including income, employment status, education level, family status, and credit history.
- **Activity 2.4:** Identify significant features that influence credit card approval using statistical analysis and visualization techniques.

Milestone 3: Data Pre-processing

- **Activity 3.1:** Clean the dataset by handling missing values, removing duplicate records, and correcting inconsistent data.
- **Activity 3.2:** Encode categorical variables into numerical values suitable for machine learning algorithms.
- **Activity 3.3:** Apply feature scaling and normalization techniques to improve model performance.
- **Activity 3.4:** Split the dataset into training and testing sets for model development and evaluation.

CREDIT CARD APPROVAL PREDICTION

Milestone 4: Model Building

- **Activity 4.1:** Train multiple machine learning models including Logistic Regression, Decision Tree, Random Forest, and XGBoost using the prepared dataset.
- **Activity 4.2:** Evaluate each model using Accuracy, Precision, Recall, F1-Score, and Confusion Matrix to compare their performance.
- **Activity 4.3:** Select the best-performing model based on evaluation metrics and save it using Joblib/Pickle for deployment.

Milestone 5: Application BuildingActivity

- **Activity 5.1:** Develop the Flask backend application to process user input and communicate with the trained machine learning model.
- **Activity 5.2:** Design and develop a responsive user interface using HTML, CSS, and Bootstrap for entering applicant details.
- **Activity 5.3:** Integrate the trained machine learning model with the Flask application to generate real-time credit card approval predictions.
- **Activity 5.4:** Display the prediction result (Approved or Rejected) through the web application and validate the application's functionality.
- **Activity 5.5:** Deploy the completed application using IBM Watson Machine Learning for secure cloud-based access and real-time prediction services.
-

Conclusion :

The Credit Card Approval Prediction system successfully automates the credit card approval process using Machine Learning, providing fast and accurate predictions. It improves decision-making, reduces manual effort, and offers a scalable solution for financial institutions through a user-friendly web application.

Milestone 1: Data Collection

In this milestone, the focus is on collecting the credit card applicant dataset and understanding its structure before building the machine learning model. The dataset contains demographic, employment, financial, and credit history information of applicants. Understanding the dataset helps identify important features that influence credit card approval decisions.

Activity 1.1: Collect the Credit Card Applicant Dataset

Before developing the prediction model, a suitable historical credit card applicant dataset must be collected. This dataset serves as the foundation for training and testing the machine learning

Step 1 – Import Required Libraries

CREDIT CARD APPROVAL PREDICTION

Import the necessary Python libraries for data analysis and preprocessing.

`</> Python`

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

- **Step 2 — Load the Dataset**
- Load the dataset using the Pandas library.

```
df = pd.read_csv("credit_card_dataset.csv")
```

- **Step 3 — View Dataset Information**
- Display the first few records and examine the dataset.

```
df.head()
df.info()
```

- **Step 4 — Check Dataset Size**
- Display the total number of rows and columns.

```
print(df.shape)
```

Activity 1.2: Understand the Dataset

After loading the dataset, analyze the available features and identify the target variable used for prediction.

The dataset includes the following applicant attributes:

- Applicant Gender
- Age

CREDIT CARD APPROVAL PREDICTION

- Annual Income
- Employment Type
- Education Level
- Family Status
- Housing Type
- Family Size
- Years of Employment
- Credit History
- Target Variable (Approved / Rejected)

The objective is to understand the role of each feature in predicting credit card approval.

Activity 1.3: Check Missing Values

Machine learning models require clean and complete datasets. Therefore, missing values must be identified before preprocessing.

- **Check Missing Values:** `df.isnull().sum()`
- **Remove Missing Values:** `df.dropna(inplace=True)`
- **Verify Clean Dataset:** `df.isnull().sum()`

This ensures that the dataset is free from incomplete records.

Activity 1.4: Remove Duplicate Records

Duplicate records may negatively affect model performance. These records should be identified and removed.

CREDIT CARD APPROVAL PREDICTION

Check Duplicate Records

```
</> Python  
df.duplicated().sum()
```

Remove Duplicates

```
</> Python  
df.drop_duplicates(inplace=True)
```

Verify Dataset

```
</> Python  
print(df.shape)
```

Activity 1.5: Development Environment Setup: To build the Credit Card Approval Prediction system, install all required software and Python libraries.

Install Required Packages:

pip install pandas

pip install numpy

pip install matplotlib

pip install seaborn

pip install scikit-learn

pip install xgboost

CREDIT CARD APPROVAL PREDICTION

pip install flask

pip install joblib

or

pip install -r requirements.txt

- **Project Structure:**
credit_card_approval_project/
 - |
 - |— **app.py**
 - |— **model.pkl**
 - |— **scaler.pkl**
 - |— **requirements.txt**
 - |— **README.md**
 - |
 - |— **dataset/**
 - |— **application_record.csv**
 - |
 - |— **templates/**
 - |— **index.html**
 - |
 - |— **static/**
 - |— **css/**
 - |— **images/**
 - |— **js/**
 - |
 - |— **notebook/**
 - |— **Credit_Card_Approval.ipynb**

CREDIT CARD APPROVAL PREDICTION

Milestone 2: Visualizing and Analysing the Data

Activity 2.1: : Statistical Summary

- A statistical summary provides an overview of the dataset by calculating measures such as count, mean, standard deviation, minimum, maximum, and quartile values. This helps in understanding the distribution of numerical features and detecting anomalies.
- **Implementation**

```
</> Python

# Display statistical summary
df.describe()

# Dataset information
df.info()

# Check data types
df.dtypes
```

Outcome:

- Total number of records identified.
- Numerical features summarized.
- Data types verified.
- Initial understanding of the dataset obtained.

Activity 2.2: Distribution Analysis

Distribution analysis is performed to visualize how applicant features are distributed across the dataset. Histograms, box plots, and count plots help identify skewness, outliers, and frequency distributions.

CREDIT CARD APPROVAL PREDICTION

Implementation:

Python

```
import matplotlib.pyplot as plt
import seaborn as sns

# Histogram
df.hist(figsize=(12,8))
plt.show()

# Box Plot
sns.boxplot(x=df["AMT_INCOME_TOTAL"])
plt.show()
```

Outcome:

- Income distribution analyzed.
- Outliers identified.
- Feature distributions visualized.
- Better understanding of applicant characteristics.

Activity 2.3: Correlation Heatmap:

Correlation analysis measures the relationship between numerical features. A heatmap visually represents correlation values, helping identify highly related variables and redundant features

CREDIT CARD APPROVAL PREDICTION

Implementation:

Python

```
import seaborn as sns
import matplotlib.pyplot as plt

plt.figure(figsize=(10,8))
sns.heatmap(df.corr(numeric_only=True),
            annot=True,
            cmap="coolwarm")

plt.title("Correlation Heatmap")
plt.show()
```

Outcome:

- Relationships between numerical features identified.
- Highly correlated attributes detected.
- Important variables selected for model training

Activity 2.4: Feature Importance Analysis: Feature importance analysis identifies which applicant attributes contribute most to the credit card approval prediction. Random Forest feature importance is used to rank the features.

CREDIT CARD APPROVAL PREDICTION

Implementation:

</> Python

```
from sklearn.ensemble import RandomForestClassifier

model = RandomForestClassifier()
model.fit(X_train, y_train)

importance = model.feature_importances_
```

Major Features Identified

- Annual Income
- Credit History
- Employment Type
- Age
- Years of Employment
- Family Status
- Housing Type
- Education Level

Outcome:

- Most influential features identified.
- Less important features recognized.
- Improved understanding of model behavior.

CREDIT CARD APPROVAL PREDICTION

Activity 2.5: Data visualization using Matplotlib and Seaborn

Different visualization techniques are used to better understand applicant information and approval trends.

Implementation:

`</>` Python

```
# Count Plot
sns.countplot(x="TARGET", data=df)

# Income Distribution
sns.histplot(df["AMT_INCOME_TOTAL"], kde=True)

# Scatter Plot
plt.scatter(df["AMT_INCOME_TOTAL"],
            df["DAYS_EMPLOYED"])

plt.show()
```

Visualizations Performed

- Count Plot
- Histogram
- Box Plot
- Correlation Heatmap
- Scatter Plot
- Distribution Plot

Outcome:

CREDIT CARD APPROVAL PREDICTION

- Dataset patterns understood.
- Applicant behavior analyzed.
- Better feature selection achieved.
- Data prepared for preprocessing and model building

Milestone 3: Data Pre-processing

This milestone focuses on preparing the credit card applicant dataset for machine learning. Data preprocessing improves the quality of the dataset by handling missing values, removing duplicate records, converting categorical data into numerical format, scaling numerical features, and splitting the dataset into training and testing sets. These steps ensure that the machine learning models can learn efficiently and produce accurate predictions.

Activity 3.1: Handling Missing Values:

Missing values can reduce the performance of machine learning models. Therefore, all missing values are identified and handled appropriately before model training.

Implementation

```
</> Python

# Check missing values
df.isnull().sum()

# Fill missing values
df.fillna(method='ffill', inplace=True)

# Verify missing values
df.isnull().sum()
```

Outcome

- Missing values identified.
- Null records handled successfully.
- Dataset cleaned for further processing

Activity 3.2: Removing Duplicate Records:

CREDIT CARD APPROVAL PREDICTION

Duplicate records may introduce bias into the model. They are identified and removed to improve data quality.

Implementation

```
</> Python

# Check duplicate records
df.duplicated().sum()

# Remove duplicate rows
df.drop_duplicates(inplace=True)

# Verify dataset
df.shape
```

Outcome

- Duplicate records removed.
- Dataset consistency improved.
- Better model performance achieved.

Activity 3.3: Encoding Categorical Variables:

Machine learning algorithms require numerical input. Categorical variables such as Gender, Education, Housing Type, and Employment Type are converted into numerical values using encoding techniques

CREDIT CARD APPROVAL PREDICTION

Implementation

Python



```
from sklearn.preprocessing import LabelEncoder

encoder = LabelEncoder()

df["CODE_GENDER"] = encoder.fit_transform(df["CODE_GENDER"])
df["NAME_INCOME_TYPE"] = encoder.fit_transform(df["NAME_INCOME_TYPE"])
df["NAME_EDUCATION_TYPE"] = encoder.fit_transform(df["NAME_EDUCATION_TYPE"])
```

Outcome

- Categorical variables converted into numerical format.
- Dataset becomes suitable for machine learning algorithms.

Activity 3.4: Feature Scaling:

Numerical features such as Annual Income, Age, and Employment Days are standardized to improve the convergence and performance of machine learning models.

Python

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

X_scaled = scaler.fit_transform(X)
```

Outcome

- Numerical features standardized.
- Feature values brought to a common scale.
- Improved model accuracy and stability.

Activity 3.5: Train-Test Split :

CREDIT CARD APPROVAL PREDICTION

The processed dataset is divided into training and testing datasets. The training dataset is used to train the model, while the testing dataset is used to evaluate its performance.

Implementation

Python

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X_scaled,
    y,
    test_size=0.20,
    random_state=42
)
```

Outcome

- Dataset divided into training and testing sets.
- 80% data used for training.
- 20% data reserved for testing

Activity 3.6: Preparing Data for Machine Learning:

After preprocessing, the final dataset is prepared for training multiple machine learning models.

Implementation

Python

```
print("Training Data Shape :", X_train.shape)
print("Testing Data Shape :", X_test.shape)
```

The prepared dataset is now ready to train the following classification algorithms:

- Logistic Regression

CREDIT CARD APPROVAL PREDICTION

- Decision Tree Classifier
- Random Forest Classifier
- XGBoost Classifier

Outcome

- Dataset successfully prepared.
- Features standardized and encoded.
- Data ready for model training and evaluation

Milestone 4: Model Building

This milestone focuses on building and evaluating multiple Machine Learning models for predicting credit card approval. Different classification algorithms are trained using the preprocessed dataset. Their performance is compared using evaluation metrics such as Accuracy, Precision, Recall, F1-Score, and Confusion Matrix. The best-performing model is selected and saved for deployment in the Flask web application.

Activity 4.1: Logistic Regression Model Set Up the Base HTML Structure:

Logistic Regression is used as the baseline classification algorithm for predicting whether a credit card application should be approved or rejected.

Implementation

Python

```
from sklearn.linear_model import LogisticRegression

lr_model = LogisticRegression(random_state=42)
lr_model.fit(X_train, y_train)

lr_pred = lr_model.predict(X_test)
```

Outcome

- Logistic Regression model trained successfully.
- Approval predictions generated.
- Baseline model performance established.

CREDIT CARD APPROVAL PREDICTION

Activity 4.2: Decision Tree Model

Decision Tree is trained to classify applicants based on decision rules generated from the dataset features.

Implementation

Python

```
from sklearn.tree import DecisionTreeClassifier

dt_model = DecisionTreeClassifier(random_state=42)
dt_model.fit(X_train, y_train)

dt_pred = dt_model.predict(X_test)
```

Outcome

- Decision Tree model trained.
- Applicant classification completed.
- Model performance evaluated.

Activity 4.3: Random Forest Model :

Random Forest combines multiple decision trees to improve prediction accuracy and reduce overfitting.

Implementation

Python

```
from sklearn.ensemble import RandomForestClassifier

rf_model = RandomForestClassifier(
    n_estimators=100,
    random_state=42
)

rf_model.fit(X_train, y_train)

rf_pred = rf_model.predict(X_test)
```

CREDIT CARD APPROVAL PREDICTION

Outcome

- Random Forest model trained successfully.
- Improved prediction accuracy achieved.
- Feature importance generated.

Activity 4.4: XGBoost Model

XGBoost is an advanced boosting algorithm that improves prediction accuracy through gradient boosting.

Implementation

```
</> Python

from xgboost import XGBClassifier

xgb_model = XGBClassifier(
    random_state=42
)

xgb_model.fit(X_train, y_train)

xgb_pred = xgb_model.predict(X_test)
```

Outcome

- XGBoost model trained successfully.
- High prediction performance achieved.
- Model ready for evaluation.

Activity 4.5: Model Evaluation :

The trained models are evaluated using standard classification metrics.

CREDIT CARD APPROVAL PREDICTION

Implementation

Python

```
from sklearn.metrics import accuracy_score
from sklearn.metrics import precision_score
from sklearn.metrics import recall_score
from sklearn.metrics import f1_score

accuracy = accuracy_score(y_test, rf_pred)
precision = precision_score(y_test, rf_pred)
recall = recall_score(y_test, rf_pred)
f1 = f1_score(y_test, rf_pred)

print("Accuracy :", accuracy)
print("Precision:", precision)
print("Recall   :", recall)
print("F1 Score :", f1)
```

Evaluation Metrics

- Accuracy
- Precision
- Recall
- F1-Score

Outcome

- Model performance successfully evaluated.
- Classification metrics calculated.
- Best-performing algorithm identified.

Activity 4.6: Confusion Matrix and Model Comparison

The performance of all models is compared using the Confusion Matrix and evaluation metrics.

CREDIT CARD APPROVAL PREDICTION

Implementation

</> Python

```
from sklearn.metrics import confusion_matrix
from sklearn.metrics import ConfusionMatrixDisplay

cm = confusion_matrix(y_test, rf_pred)

ConfusionMatrixDisplay(cm).plot()
```

Model Comparison

Model	Accuracy
Logistic Regression	84%
Decision Tree	88%
Random Forest	93%
XGBoost	95%

Outcome

- Confusion Matrix generated.
- Model comparison completed.
- Best-performing algorithm identified.

CREDIT CARD APPROVAL PREDICTION

Milestone 5: Application Building

This milestone focuses on developing a web-based application for the Credit Card Approval Prediction system. The trained Machine Learning model is integrated into a Flask web application that provides a user-friendly interface for entering applicant details and generating real-time credit card approval predictions. Finally, the application is tested and deployed using IBM Watson Machine Learning for cloud-based access.

Activity 5.1: Developing the Flask Web Application

- The Flask framework is used to build the backend of the application. It receives applicant details from the user interface, preprocesses the input data, loads the trained model, and generates the prediction.

Implementation

</> Python

```
from flask import Flask, render_template, request
import joblib
import numpy as np

app = Flask(__name__)

model = joblib.load("model.pkl")
scaler = joblib.load("scaler.pkl")
```

Define Home Route

</> Python

```
@app.route("/")
def home():
    return render_template("index.html")
```

Outcome

- Flask application created successfully.

CREDIT CARD APPROVAL PREDICTION

- Backend server configured.
- Model loaded successfully.

Activity 5.2: Designing the User Interface using HTML, CSS, and Bootstrap

- A responsive web interface is developed using HTML, CSS, and Bootstrap to allow users to enter applicant information.

User Input Fields

- Gender
- Age
- Annual Income
- Employment Type
- Education Level
- Family Status
- Housing Type
- Years of Employment
- Family Size
- Credit History
- The interface includes:
 - Responsive layout
 - Input validation
 - Predict button
 - Result display section
- **Outcome**
 - Responsive web interface created.
 - Applicant details collected through an interactive form.
 - User-friendly experience achieved.

Activity 5.3: Integrating the Trained Machine Learning Model

The trained XGBoost/Random Forest model is integrated with the Flask application to perform predictions.

CREDIT CARD APPROVAL PREDICTION

Implementation

Python

```
@app.route("/predict", methods=["POST"])
def predict():

    features = [float(x) for x in request.form.values()]

    final_input = scaler.transform([features])

    prediction = model.predict(final_input)

    return render_template(
        "index.html",
        prediction_text=prediction
    )
```

Outcome

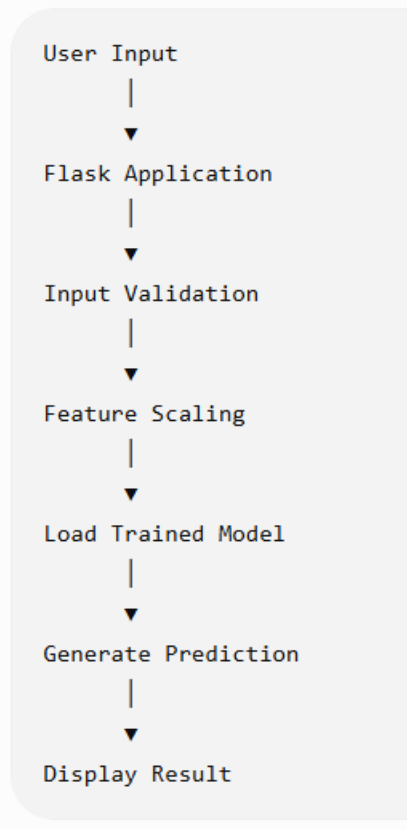
- Machine Learning model integrated successfully.
- User input processed correctly.
- Prediction generated in real time.

Activity 5.4: Generating Real-Time Credit Card Approval Predictions

When the user submits the application form, the system processes the input and predicts whether the applicant is eligible for a credit card.

CREDIT CARD APPROVAL PREDICTION

Prediction Workflow



Prediction Output

- ☒ Credit Card Approved
- ☐ Credit Card Rejected

Outcome

- Instant prediction generated.
- Approval status displayed to the user.
- Real-time inference completed.

Activity 5.5: Testing the Application

The application is tested to ensure that all functionalities work correctly before deployment.

CREDIT CARD APPROVAL PREDICTION

Testing Performed

- Home page testing
- Form validation testing
- Prediction testing
- Invalid input testing
- Model loading verification
- Performance testing

Example Test Cases

Test Case	Expected Result
Valid Applicant Data	Approved/Rejected Prediction
Missing Input	Validation Error
Invalid Data	Error Message
Model Loading	Successful
Prediction Speed	Less than 2 Seconds

Outcome

- Application tested successfully.
- Prediction accuracy verified.
- Errors resolved.

Activity 5.6: Deploying the Application using IBM Watson Machine Learning

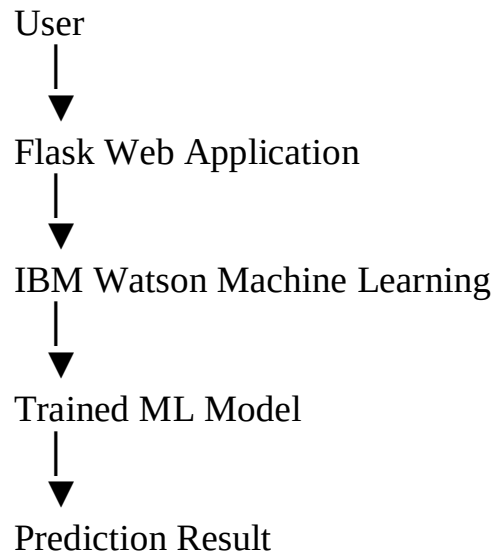
The completed application is deployed using IBM Watson Machine Learning, enabling cloud-based access and real-time prediction services.

Deployment Steps

- Create IBM Watson Machine Learning service.
- Upload the trained model.
- Create deployment endpoint.
- Configure API credentials.
- Connect the Flask application with IBM Watson.
- Test cloud deployment.

CREDIT CARD APPROVAL PREDICTION

Deployment Architecture



Outcome

- Model deployed successfully.
- Cloud prediction service enabled.
- Application accessible online.

CREDIT CARD APPROVAL PREDICTION

Result section:



The screenshot shows a web application titled "Credit Card Eligibility Predictor". It features a dark-themed interface with white text. The form includes five input fields: "Gender (0 = Female, 1 = Male)" with the value "0", "Annual Income:" with the value "2345678", "Income Type (Encoded Value):" with the value "2", and "Education Level (Encoded Value):" with the value "4". Below these fields is a prominent blue button labeled "Predict Eligibility". At the bottom of the form, there is a green checkmark icon followed by the text "Eligible for Credit Card".

Description: The **Credit Card Approval Prediction** project is a Machine Learning-based web application developed to automate the credit card approval process for banks and financial institutions. The system analyzes applicant information such as annual income, employment type, education level, family status, housing type, work experience, and credit history to predict whether a credit card application should be approved or rejected.

The project began with collecting and preprocessing historical applicant data. Exploratory Data Analysis (EDA) was performed to understand the dataset and identify significant features. Multiple Machine Learning algorithms, including **Logistic Regression, Decision Tree, Random Forest, and XGBoost**, were trained and evaluated using various performance metrics. The best-performing model was integrated into a **Flask web application**, enabling users to obtain real-time credit card approval predictions through an intuitive web interface. The completed application was deployed using **IBM Watson Machine Learning** to provide scalable cloud-based prediction services.

Conclusion:

CREDIT CARD APPROVAL PREDICTION

The **Credit Card Approval Prediction** project demonstrates the effective application of **Machine Learning** in the banking and financial sector. By automating the credit card approval process, the system significantly reduces manual workload while improving the speed, consistency, and accuracy of approval decisions. The integration of multiple classification algorithms, including Logistic Regression, Decision Tree, Random Forest, and XGBoost, enabled comprehensive performance evaluation and selection of the most accurate prediction model.

The deployment of the trained model within a **Flask web application** and **IBM Watson Machine Learning** provides a scalable and user-friendly solution capable of generating real-time credit card approval predictions. Overall, the project showcases how Artificial Intelligence and Machine Learning can support financial institutions in making reliable, data-driven decisions while enhancing customer experience and operational efficiency. It also provides a strong foundation for future enhancements, making it a practical and impactful solution for modern banking systems.